

Common Distributed Processing Library User Guide

V-2023.09-SP2

December 2023

For questions or comments, contact the support team for the product under which this component is invoked.



Copyright Notice and Proprietary Information

© 2023 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

TABLE OF CONTENTS

WHAT'S NEW IN CDPL.....	4
Disable All CDPL Logs	4
Dpcheck Utility Program Enhanced	4
Problem Report API Enhanced	4
CDPL USAGE.....	5
INTRODUCTION	5
OPERATIONAL ENVIRONMENT	6
INFRASTRUCTURE SPECIFICATION FOR DISTRIBUTED PROCESSING	12
HOSTS INFORMATION FILE	13
USING THE HOSTS INFORMATION FILE TO SPECIFY RSH RUNS.....	15
USING THE HOSTS INFORMATION FILE TO SPECIFY SSH RUNS	15
USING THE HOSTS INFORMATION FILE TO SPECIFY SH RUNS.....	16
USING THE HOSTS INFORMATION FILE TO SPECIFY SGE RUNS	16
USING THE HOSTS INFORMATION FILE TO SPECIFY LSF RUNS.....	18
USING THE HOSTS INFORMATION FILE TO SPECIFY PBS RUNS	19
USING THE HOSTS INFORMATION FILE TO SPECIFY RTDA RUNS	20
USING THE HOSTS INFORMATION FILE TO SPECIFY NETBATCH RUNS.....	21
USING THE HOSTS INFORMATION FILE TO SPECIFY SLURM RUNS.....	22
USING THE HOSTS INFORMATION FILE TO SPECIFY CUSTOM SUBMIT COMMAND	23
USING THE HOSTS INFORMATION FILE TO SPECIFY CUSTOM LAUNCH PROTOCOL.....	24
WORKER INFORMATION FILE.....	27
DOCKER CONFIGURATION FILE	29
DEBUGGING DISTRIBUTED ENVIRONMENTS	31
DPCHECK – INFRASTRUCTURE VALIDATION	31
Checking a Hosts Information File With dpcheck	33
Checking SGE Settings With dpcheck.....	33
Checking LSF Settings With dpcheck.....	33
Checking PBS Settings With dpcheck	34
Checking RTDA Settings With dpcheck	34
Checking Netbatch Settings With dpcheck	34
Example: An Erroneous SGE Configuration.....	35
DPVALIDATE – INFRASTRUCTURE VALIDATION	35
DP ANALYZER.....	36
LOG FILES	38
CDPL Release Version	38
LOG VIEWING UTILITIES: LOG2TTY AND LOG2HTML.....	38
Sample Log From a Successful Run	40
Sample Log With Errors	41
MANAGING DP LOG FILES: DPPURGE	41

What's New in CDPL

The following are the new features, enhancements, and changes available in the V-2023.09 release of CDPL.

Disable All CDPL Logs

Setting CDPL_LOGDIR to /dev/null can disable all CDPL logs if the EDA tool has not defined CDPL_LOGDIR's value.

Dpcheck Utility Program Enhanced

The dpcheck utility program is enhanced based on user requests.

Problem Report API Enhanced

CDPL's problem report API is enhanced to cover issues that happen at distributed processing start-up time.

1

CDPL Usage

Introduction

Common Distributed Processing Library (CDPL) is an internal library used by Synopsys products to activate **parallel and distributed processing** of EDA engineering computations. CDPL provides core building blocks such as:

- DP Scheduler (DPS) that facilitates task formation and dispatching, traditionally known as DP master.
- DP Unit (DPU) that facilitates the construction of task execution engine, also known as DP worker.

CDPL offers such functions through its API.

With CDPL, Synopsys EDA tools can:

- Build a full-fledged DP (distributed processing) system with centralized job controller(s) and distributed remote execution engines.
- Interact with familiar or customized grid engines (for procuring compute resources).

From an end-user point of view, Synopsys EDA tools use CDPL to:

- Launch DP workers.
- Schedule tasks.
- Manage the processing of tasks.

CDPL also includes a DP Manager GUI program for tracking job status and viewing DP job processing details.

The following sections of this guide provide information on the UNIX environment variables used by CDPL and DP infrastructure setup:

- [Hosts Information File](#)
- [Worker Information File](#)
- [Docker Configuration File](#)
- [LSF Interface](#)
- [SGE Interface](#)
- [PBS Interface](#)
- [RTDA Interface](#)
- [Netbatch Interface](#)
- [SLURM Interface](#)
- [Customized Submit](#)
- [CUSTOM Protocol](#)

Operational Environment

Advanced users¹ can use the following CDPL environment variables to control or change the default behavior of the CDPL:

`CDPL_HOME`: <REQUIRED>: The root directory of the CDPL-related executables and libraries.

`CDPL_LOGDIR`: <OPTIONAL>: The directory to write the CDPL log files. If the directory is not specified, the default is to use the current working directory.

`CDPL_ARRAY_JOBS_SUBMIT_RANDOMIZED`: <OPTIONAL>: Set this variable to allow CDPL to submit a job array command through one of the workers in the array. By default, CDPL always submits a job array command through the first worker in the array. The impact of this variable should be minimal because even the first worker in the job array is placed on a randomly picked host on a compute farm.

`CDPL_ARRAY_WORKERS_EXIT_TOGETHER`: <OPTIONAL>: Set this variable to allow CDPL to treat all *pending* jobs submitted in a job array command as failed if the *submitted* job fails. For example, if you submit the first 1-10 jobs as a job array by using a single grid submit command and if the first job fails after being successfully submitted, all the other nine pending jobs are

¹ Synopsys EDA tools users are recommended to consult the tools' user manuals first for usage, configuration, set up, and troubleshooting purposes before trying to use CDPL environment variables directly.

considered as failed. By default, the failure of the submitted job does not have an impact on the other *pending* jobs (in the array), which can be still pending on the grid engine or have failed.

CDPL_BCASTDIR: <OPTIONAL>: The location to write the CDPL broadcast files. When unset, the default location `$HOME/.synopsys/cdpl` is used. The broadcast files are created by CDPL and are used by the DP Manager GUI to monitor and display information about running DP jobs.

CDPL_CONSECUTIVE_FAILURES_DELAY: <OPTIONAL>: Set this variable to define how long CDPL should pause the job submission after a given number of consecutive job-submit failures (see **CDPL_MAX_CONSECUTIVE_FAILURES**). Its value should be set in seconds. For example, setting a value to 5 means CDPL should pause for five seconds before the new job submission.

CDPL_CUSTOM_SUBMIT: <OPTIONAL>: Set this variable to 1 to start workers with a non-standard (custom) command. The standard commands are UNIX shell commands such as `rsh` and `ssh`; and known grid engines' batch job submit command such as `bsub` and `qsub`.

CDPL_CUSTOM_SUBMIT_OPTIONS: <OPTIONAL>: When using a non-standard (custom) command to start workers, this environment variable specifies a string of options to pass with the custom command. This feature is currently supported for the `SSH` protocol only.

CDPL_DEBUG_SUBMIT_TIMEOUT: <OPTIONAL>: CDPL has a default value of 60 seconds for the farm job `submit` command to timeout by itself. In some cases, such as running Purify or debugging, setting a large value (in seconds) to this environment variable overrides the default – preventing unnecessary timeout. Notice that this timeout only applies to how soon a submit command should complete by itself but does not apply to how soon a (remote) worker should come online.

CDPL_DEBUG_FARM_SUBMIT: When this environment variable is set to 1, the CDPL farm job submit processes `stdout` and `stderr` that are saved in a log file, named `master.<hostname>.<pid>.dbg`. Without this environment variable setup, CDPL only logs the `stderr` job submit process in a file named `master.<hostname>.<pid>.err`, but the `stdout` process is redirected to `/dev/null`.

`CDPL_DOCKER_CONFIG`: <OPTIONAL>: Use this environment variable to set or override the Docker configuration of the application when launching DP Units (or DP workers) in a Docker container environment on remote machines. This environment variable takes the full name of the Docker configuration file as its value.

`CDPL_FARM_SUBMIT_DONT_CAPTURE_JOBID`: <OPTIONAL>: Set this variable to prevent CDPL from waiting and capturing the job ID returned by the grid engine. By default, when submitting array jobs, CDPL waits and captures the job ID that is returned by the grid engine. This variable impacts only the job array submit cases.

`CDPL_FARM_WORKER_CHECK_DISABLE`: <OPTIONAL>: Set this environment variable to deactivate CDPL's periodic checking on the status of the submitted and pending jobs on the compute grid. This environment variable is applicable for SGE/LSF/RTDA/SLURM types of grids.

`CDPL_FARM_WORKER_CHECK_INTERVAL`: <OPTIONAL>: Use this environment variable to change the value of the default checking interval, which is 600 seconds (that is 10 minutes). If a very small value is set, the actual farm-worker checking interval will also get limited by other constraints and might take longer than the specified value.

Note: Setting a very small value might incur a negative impact on the performance.

`CDPL_FARM_WORKER_TIMEOUT`: <OPTIONAL>: Use this environment variable to define a timeout limit (in seconds) after a DP worker is submitted into the compute grid and before the worker's registration at the DP scheduler. At the set timeout limit, the DP scheduler will mark a submitted job as having timed out and will discard its internal data structure. When this environment variable is not set, CDPL does not have a timeout limit for workers submitted to compute grids. This does not impact the farms constructed with the SH, RSH, or SSH protocol.

`CDPL_FARM_WORKER_CLEANUP_ALL`: <OPTIONAL>: You require this variable for SGE, LSF, and PBS Professional grids. You can use this variable along with the `CDPL_FARM_WORKER_TIMEOUT` variable to timeout a submitted job and cancel the job queued on the grid. In this case, set its value to 0 (zero).

`CDPL_GRID_CHECK_NOREPLY_OKAY`: <OPTIONAL>: On the SGE/LSF/SLURM grids, if CDPL queries the grid engine about the status of a

set of submitted jobs but gets an empty report due to the grid engine's configuration or business, by default CDPL will move on without doing anything to the queried jobs. However, if you need to interpret an empty report as the jobs in the query are just no longer on the grid, set this environment variable to 0 (zero) to override the default CDPL behavior. That is, tell CDPL to mark those jobs as exited. This environment variable is not applicable to RTDA grids. For the RTDA grid, CDPL always assumes the default behavior.

`CDPL_GRID_CHECK_USE_HOSTCFG_TMPDIR`: <OPTIONAL>: This environment variable is applicable to SGE/LSF/RTDA/SLURM type of grids. It is used when the application provides a temporary directory in the host configuration and stores the farm worker checking (that is, grid job checking) results into that temporary directory. The application is responsible for cleaning up the content inside the temporary directory after the run.

`CDPL_MAX_CONSECUTIVE_FAILURES`: <OPTIONAL>: Set this variable to define the number of consecutive job-submit failures allowed before CDPL should pause submitting jobs further. For example, setting a value to 2 means after two consecutive job-submit failures, stop job submission until a delay time passes. See `CDPL_CONSECUTIVE_FAILURES_DELAY` to set a delay (pause) time.

`CDPL_MONITOR_FLUSH_RATE`: <OPTIONAL>: Set this variable to modify the CDPL monitor API's data collection. For example, sampling frequency. The default frequency is one sampling every five seconds.

`CDPL_SECURITY_STRICT`: <OPTIONAL>: By default, CDPL ignores an IPC connection request from unknown sources (such as port scanning software). If you want to enforce a stricter role on zero tolerance of unknown IPC requests, set this environment variable with any value (for example 0 or 1) to shut down DP as soon as an unknown request is received. For CDPL CPP API users, this environment variable is the only way for overriding the CDPL default setting.

`CDPL_SEQUENTIAL_GRID_SUBMIT`: <OPTIONAL>: When this environment variable is set, CDPL submits grid jobs one after another in a sequence. When this environment variable is not set, by default, CDPL submits several grid jobs simultaneously. Setting this environment variable has no impact on farms constructed with the SH, RSH, or SSH protocol.

CDPL_SERVER_IPADDR: <OPTIONAL>: When this environment variable is set, the CDPL IPC server uses the given IP address. This might be required in a multi-host environment.

CDPL_SERVER_PORT: <OPTIONAL>: When this environment variable is set, the CDPL IPC server binds the given port number or a random port number inside a port range. Use this environment variable to cooperate with the firewall roles of the consumers. The following syntax is used in the C shell environment:

- For individual ports
`setenv CDPL_SERVER_PORT <port number>,<port number>`
- For the port range
`setenv CDPL_SERVER_PORT <port number>-<port number>`

For example:

1. `setenv CDPL_SERVER_PORT 50000`
This setting binds the IPC server to port 50000.
2. `setenv CDPL_SERVER_PORT 50000,60000,65000`
This setting binds the IPC server to a port number randomly picked among 50000, 60000, or 65000.
3. `setenv CDPL_SERVER_PORT 50000,58000-58500`
This setting binds the IPC server to a port number randomly picked among 50000 and between 58000 and 58500, inclusive.
4. `setenv CDPL_SERVER_PORT 50000-51000,58000-58500`
This setting binds the IPC server to a port number that is randomly picked between 50000 and 51000, inclusive; or between 58000 and 58500, inclusive.

CDPL_SPURIOUS_EPOLLOUT_EVENTS_THRESHOLD: <OPTIONAL>: Use this environment variable to add fault-tolerance in handling temporary IPC send issues. If this environment variable is not set, CDPL will log a warning message each time when spurious EPOLLOUT happens but does not error out. Setting this environment variable to a positive integer defines how many spurious EPOLLOUTs are tolerable within one second. When the spurious EPOLLOUT event counts pass that number (the threshold), CDPL will error out. However, if this environment variable is set to -1 (i.e., unlimited), CDPL will not error out in spurious EPOLLOUT cases but will insert a delay in the IPC event handling loop, expecting that the system in this manner might recover from temporary issues.

CDPL_SUBMIT_FAILURE_THRESHOLD: <OPTIONAL>: Use this environment variable to increase the CDPL compute farm/grid job submit failure threshold. By default, if it fails to submit a job onto a compute farm/grid, CDPL will mark that farm/grid as unusable. Setting this environment variable with a larger value allows more submission failures to occur before CDPL disables the compute farm/grid. When a farm/grid is marked as disabled by CDPL, job submission will not happen to that farm/grid in that DP run.

CDPL_TCP_KEEPAIVE: <OPTIONAL>: Use this environment variable to turn on TCP keepalive. TCP keepalive helps to avoid the half-open socket problem. The value for this environment variable can be set to 1.

CDPL_TCP_KEEPAIVE_INTERVAL: <OPTIONAL>: Use this environment variable to override default TCP keepalive interval, which is two hours (i.e., 7200 seconds). The value for this environment variable should be in seconds. When this environment variable is set, it is not necessary to set CDPL_TCP_KEEPAIVE also, to reduce the number of environment variables.

CDPL_VFORK_JOB_SUBMIT: <OPTIONAL>: Use this environment variable to override the CDPL default job submit to avoid the out-of-memory issue when a consumer application has a large memory footprint. This environment variable takes two sets of values: (1) value "1" or "always" (double-quotes not part of the value) tells CDPL to submit jobs using `vfork()`. (2) value "2" or "optional" (double-quotes not part of the value) tells CDPL to submit jobs using the default method, and if that fails, automatically retry the submit using `vfork()`.

Note: The compute farm (or farm) and compute grid (or grid) are often interchangeable. A compute farm often refers to any cluster of distributed computers from which CDPL negotiates and acquires resources using one of the CDPL-supported farm protocols, such as RSH, SSH, SGE, LSF, and so on. Whereas a compute grid often refers to only those computer clusters managed by a grid engine, such as SGE, LSF, and so on.

Infrastructure Specification for Distributed Processing

For a distributed processing system that uses scale-out solutions, you must specify how remote processes are launched. This is the **launch protocol** in CDPL terminology. Also, you must indicate how many DP workers or agents can be used during a given run. This is the **capacity** of the distributed run in CDPL terminology.

An EDA tool is recommended to provide its own setup/configuration files for defining the launch protocol and DP capacity, then pass the settings to the run-time system of the CDPL through CDPL API calls. Alternatively, if an EDA tool exposes the *launch protocol* through the CDPL **hosts information file** and *DP capacity* through the CDPL **worker information file** to end users, the following information should be verified for setting up the DP run-time system.

CDPL supports the following protocols for launching workers on remote nodes/machines:

- RSH [Remote Shell]
- SSH [Secure Shell]
- SGE [Oracle Grid Engine, originally Sun Grid Engine]
- LSF [Load Sharing Facility, from Platform Computing]
- SH [Shell process on the local host]
- PBS [Portable Batch System]
- RTDA [Runtime Design Automation Network Computer]
- Netbatch [Intel Netbatch Compute Farm]
- SLURM [Simple Linux Utility for Resource Management]
- CUSTOM [Customized protocol unknown to CDPL in advance]

These methods cover most of the consumer environments for distributed processing. Additional protocols are currently not supported.

Note: The information, for example, the required or optional data, specified for each of the protocols mentioned above varies.

2

Hosts Information File

The hosts information file allows the EDA tool user to specify machine resources for a DP run. It includes the resource acquisition command (also known as the job *submit* command) and its options such as `qsub` and its command line options, as well as logistic constraints such as the maximum number of computing slots (for example, CPU cores) to be acquired.

The hosts information file is an ASCII file. Single-line comments are allowed and must be prefixed with '#'. Every line (one entry) in the file has the following format:

```
<flag>| <hostname>| <slots>|<tmpDir>|<protocol>|<command>
```

Note: All fields except the last field are delimited with '|'.

The hosts information file can have one active entry (see the “*Flag*” field in the table that follows) for grid job submission, and multiple active entries for RSH/SSH job submission. Mixed modes of the grid and RSH/SSH submits are not supported through the hosts information file.

Field	Value	Purpose
<i>Flag</i>	Integer – can be ‘0’ or ‘1’	A ‘0’ value disables the entry (inactive) and a ‘1’ value enables the entry (active).
<i>Hostname</i>	String, valid hostname	The hostname of a valid machine where DP worker(s) can be launched. This is applicable for DP through RSH and SSH launching modes. For SGE, LSF, PBS, RTDA, Netbatch, SLURM and CUSTOM, this field should be empty. For SH, this field is ignored.
<i>Slots</i>	Integer – any valid integer	This number denotes the number of computing slots available per host (for RSH, SSH, and SH modes), or on the grid (for LSF, SGE, PBS, RTDA, Netbatch, SLURM, and CUSTOM modes). The purpose of this field is to prevent users from overloading or oversubscribing a given host in the SH, RSH, and SSH launching modes. For grid launching modes, grid engines handle workload per host thus this

		field is often set to -1, denoting “unlimited”. Note that the DP scalability and the license count limit are directly controlled by the size of DP workers defined in the worker information file. For information, see Chapter 3, Worker Information File.
<i>Temporary directory</i>	String – a valid, writable directory	Traditionally, this directory is used for storing the local data of the DP worker (if required). An application’s DP worker program can get the temporary directory name from the <code>CDPL_TMPDIR</code> environment variable.
<i>Protocol</i>	String identifier	This string ID signifies the <i>mode</i> or <i>protocol</i> used to access remote machines for DP or to launch DP jobs on the computing grid. Valid IDs are RSH, SSH, SH, SGE, LSF, PBS, RTDA, NB, SLURM, and CUSTOM.
<i>Command</i>	String – the connection command	The given string denotes the actual command string used to access remote machines for DP or to launch DP jobs on the grid.

Note:

1. When launching a worker on a compute farm, CDPL starts the remote worker in a child process and redirects the `stderr` child process to the `master.<host>.<pid>.err` file of the CDPL, and `stdout` to `/dev/null`. For debugging purposes, override such CDPL default I/O redirections by including the I/O redirection options of the farm submit command in the host configuration file. For example, SGE/UGE has a command line option `-o` for defining the `stdout` log file, and option `-e` for defining the `stderr` log file.
2. When you enter a job name in host configuration through a farm submit command-line option, the job name should not contain whitespace character(s).
3. The command field is for specifying job submitting commands and options. Do not include the DP worker program in the submit command and options.

Using the Hosts Information File to Specify RSH Runs

To run DP on known hosts using the RSH protocol, use the following format in the hosts information file:

```
# HOSTS INFORMATION FILE
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol | command
#
1| teralab-x15 | 4 | /remote/users/tmp | RSH | rsh
1| teralab-x16 | 8 | /remote/users/tmp | RSH | rsh
```

This information file specifies a 12-worker, 2-node configuration for a DP run. Note that these are the maximum workers that will be launched. Specifically, CDPL will launch up to four workers on `teralab-x15` and up to eight workers on `teralab-x16`.

Using the Hosts Information File to Specify SSH Runs

To run DP on known hosts using the SSH protocol, use the following format in the hosts information file:

```
# HOSTS INFORMATION FILE (SSH)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol | command
#
1| cdpl_lab-x15 | 4 | /remote/users/tmp | SSH | ssh
1| cdpl_lab-x16 | 8 | /remote/users/tmp | SSH | ssh
```

This information file specifies a 12-worker, 2-machine SSH infrastructure.

CDPL with SSH requires password-less login. The following are the steps to set up the password-less login for SSH.

```
% cd ~/.ssh
% ssh-keygen -f id_rsa -N '' -t rsa -q
[You do not need a pass phrase.]
% cp id_rsa.pub authorized_keys
```

Using the Hosts Information File to Specify SH Runs

To run DP on a localhost by using the SH protocol, use the following format in the hosts information file:

```
# HOSTS INFORMATION FILE
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots|tmpDir| protocol | command
#
1| localhost | 8 | /remote/users/tmp | SH | sh
```

This information file specifies up to eight workers on the localhost. When using the SH protocol, you do not need to set the *hostname* field because all the workers are run on the local host. This information file sets the hostname to “localhost” as a reminder.

Use the SH protocol when RSH and SSH protocols are not allowed or not optimal and you still want your application to use CDPL to distribute the processing over multiple cores on the same machine.

Traditionally, DP workers submitted through the SH protocol are detached from the `main()` and run as orphan processes. Alternatively, the attached-submit mode keeps parent-child process relationships among `main()` and DP workers on the host. To use the attached-submit mode, replace `sh` of the command field in the host configuration with a hyphen, `-`. For example:

```
1| localhost | 8 | /remote/users/tmp | SH | -
```

Using the Hosts Information File to Specify SGE Runs

To run DP on the Sun Grid Engine (SGE) managed compute farm, use the following format:

```
# HOSTS INFORMATION FILE (SGE)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting SGE jobs
1|  | 3 | /remote/users/tmp | SGE | qsub -P bnormal -l arch=glinux
```

Note: The ‘*hostname*’ field is left empty for SGE.

This example specifies three worker slots on Linux target systems. To specify an unlimited number of workers, set the `slots` field to -1. Note that CDPL does not limit how many workers can land on each host on the SGE farm. Use the other SGE `qsub` command-line options, such as memory size, to send exclusive resource requests.

With the addition of SGE array job support to CDPL, you can now specify the `-t` option in the `qsub` command line and submit worker jobs as an array job. To use this feature, comply with the following:

- a. The start index must be 1.
- b. The end index must not exceed the maximum number of workers, that is `-t 1-N` where $N \leq \text{max workers}$.

This feature is available since N_2017.09_SP2_PROD_CDPL (December 2017) release.

```
1 |  | -1 | /tmp | SGE | qsub -P bnormal -l arch=glinux,os_version='WS6.0|SUSE11.0' -t 1-20
```

To verify if the SGE installation is valid and ready to be used, use the [dpcheck](#) utility script.

Note: SGE is the name for the original Sun Grid Engine. The same open-sourced grid engine is now called UGE. For CDPL users, UGE and SGE are the same, even though the existing submit protocol name remains SGE.

Using the Hosts Information File to Specify LSF Runs

To run DP on LSF managed compute farm, use the following format:

```
# HOSTS INFORMATION FILE (LSF)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting LSF jobs
1|   | -1 | /remote/users/tmp | LSF | bsub -app batch
```

Note: The '*hostname*' field is left empty for LSF. This example requests for an unlimited number of worker slots on LSF.

To request for a given number of slots on an LSF farm, use the following format:

```
# HOSTS INFORMATION FILE (LSF)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol#| command
# The end-user environment should be properly configured and capable of
# starting LSF jobs
1|   | 3 | /remote/users/tmp | LSF | bsub -app batch_normal
```

This example specifies a three-slot farm capacity to use LSF. In this case, no more than three workers *at any point in time* will be running.

Note that “3” does not apply to the accumulative number of workers in a DP run. Rather, when there is a worker crash and automatic worker replacement, the total number of workers *in the whole duration of* the DP run can be more than 3.

With the addition of LSF array job support to CDPL, you can specify the `-J` option in the `bsub` command line and submit the worker jobs as an array job. To use this feature, comply with the following:

- a. The start index must be 1.
- b. The end index must not exceed the maximum number of workers, that is, `-J <JobName> [1-N]` where $N \leq \text{max workers}$.

This feature is available since Q_2020.06_SP1_PROD_CDPL (Jun 2020) release.

```
1 | | -1 | /tmp | LSF | bsub -app batch_normal -J <JobName>[1-20]
```

Using the Hosts Information File to Specify PBS Runs

To run DP on PBS managed compute farm, use the following format:

```
# HOSTS INFORMATION FILE (PBS)
# Each valid host definition occupies an entire line.
# Line with '#' as the first character is a comment line.
# Format of each line:
# flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting PBS jobs
1| | 16 | /remote/users/tmp | PBS | qsub -q bnormal -l arch=glinux
```

Note: The '*hostname*' field is left empty for PBS. This example specifies 16 worker slots on the Linux target systems.

CDPL support for PBS is validated on **PBS Professional** (that is, PBS Pro). Other PBS versions that use different command line syntax are not supported by this protocol. However, the PBS job submits might be done through a CDPL custom submit or the CUSTOM protocol (see [Custom protocol](#)).

With the addition of PBS Professional array job support to CDPL, you can now specify the `-J` option in the `qsub` command line and submit the worker jobs as an array job. To use this feature, comply with the following:

- a. The start index must be 1.
- b. The end index must not exceed the maximum number of workers, that is, `-J 1-N` where $N \leq \text{max workers}$.

This feature is available since Q_2020.06_SP1_PROD_CDPL (Jun 2020) release. A range with a stepping factor (supported by PBS Professional) is not supported in CDPL.

```
1 | | -1 | /tmp | PBS | qsub -q bnormal -J 1-20
```

To verify if the PBS installation is valid and ready to use, use the [dpcheck](#) utility script.

Using the Hosts Information File to Specify RTDA Runs

To run DP on Runtime Design Automation (RTDA) managed compute farm, use the following format:

```
# HOSTS INFORMATION FILE (RTDA)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting RTDA jobs
1|  | 10 | /remote/users/tmp | RTDA | nc run
```

Note: The '*hostname*' field is left empty for RTDA. This example specifies 10 worker slots.

Valid RTDA command parameters can follow the `nc run` command string.

To verify if the RTDA installation is valid and ready to be used, use the [dpcheck](#) utility script.

The RTDA `nc` command might conflict with the Linux `/usr/bin/nc` command in the user environment. If required, specify the full path in the RTDA `nc` command. The following example is similar to the previous example except that it specifies the full path in the RTDA `nc` command.

```
# HOSTS INFORMATION FILE (RTDA)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting RTDA jobs
1|  | 10 | /remote/users/tmp | RTDA | /remote/fsd/rtda/2011.03/linux/scripts/nc run
```

Full Path to nc Command

If you submit an EDA application program, that is, `main()` function to the RTDA farm and configure the application to start DP workers through RTDA, then add the `-e "SNAPSHOT"` option to the `nc run` command to start the DP workers with the correct environment settings.

The following example describes a configuration where the DP workers are started with the `-e "SNAPSHOT"` option.

```
# HOSTS INFORMATION FILE (RTDA)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag | hostname | slots | tmpDir | protocol | command
# The end-user environment should be properly configured and capable of
# starting RTDA jobs
1 |  | 10 | /remote/users/tmp | RTDA | nc run -e "SNAPSHOT"
```

Command with Environment Snapshot Option

Note:

In the older CDPL releases, add the escape character, that is, `\` to options that have double quotes. For example, `\ "option\"`.

Using the Hosts Information File to Specify Netbatch Runs

To run DP on the Intel Netbatch-managed compute farm, use the following format:

```
# HOSTS INFORMATION FILE (Netbatch)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag | hostname | slots | tmpDir | protocol | command
# The end-user environment should be properly configured and capable of
# starting Netbatch jobs
1 |  | 8 | /remote/users/tmp | NB | nbjob run
```

The protocol field for a Netbatch hosts information file is **NB**. The '*hostname*' field is left empty for Netbatch. This example specifies 8 worker slots.

Valid Netbatch command parameters can follow the `nbjob run` command string. To verify if the Netbatch installation is valid and ready to be used, use the [dpcheck](#) utility script.

Note:

In older CDPL releases, add the escape character, that is, `\` to options that have double quotes. For example, `\ "option\"`.

Starting with the U-2022.12 release, CDPL removes Netbatch option `-mode interactive`.

Using the Hosts Information File to Specify SLURM Runs

To run DP on the SLURM-managed compute farm, use the following format:

```
# HOSTS INFORMATION FILE (SLURM)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting SLURM jobs
1|  | -1 | /remote/users/tmp | SLURM | sbatch
```

Note: The '*hostname*' field is left empty for SLURM. This example requests for an unlimited number of worker slots on SLURM.

To request a given number of slots on a SLURM farm, use the following format:

```
# HOSTS INFORMATION FILE (SLURM)
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots|tmpDir| protocol#| command
# The end-user environment should be properly configured and capable of
# starting SLURM jobs
1|  | 3 | /remote/users/tmp | SLURM | sbatch
```

This example specifies a three-slot farm capacity for using SLURM. No more than three workers *at any point in time* will be running in this case.

Note that “3” does not apply to the accumulative number of workers in a DP run. Rather, when there is a worker crash and automatic worker replacement,

the total number of workers *in the whole duration of* the DP run can be more than 3.

With the addition of SLURM array job support to CDPL, you can now specify the `-a` or `--array` option in the `sbatch` command line and submit the worker jobs as an array job. To use this feature, comply with the following:

- a. The start index must be 1.
- b. The end index must not exceed the maximum number of workers, that is, `-a 1-N` or `--array=1-N` where $N \leq \text{max workers}$.

```
1 |   | -1 | /tmp | SLURM | sbatch -a 1-4
1 |   | -1 | /tmp | SLURM | sbatch --array=1-4
```

Note:

The native SLURM support is available since the R_2020.09_SP2_PROD_CDPL (Dec 2020) release.

Using the Hosts Information File to Specify Custom Submit Command

You can use one of the launch protocols described previously with a customized, yet compatible, `submit` command to launch DP workers. This feature is known as custom submit.

The `custom submit` command has the following restrictions:

1. The command must use one of the protocols that CDPL supports, for example, RSH, SSH, SH, LSF, SGE, PBS, RTDA, SLURM, or Netbatch.
2. The custom command must be a command that is compatible with the original command.
3. To use the LSF and SGE launch protocols, the command must support the way CDPL submits the worker job, which is through the standard input.

Perform the following steps to set up a custom DP worker submit command:

1. Set the environment variable `CDPL_CUSTOM_SUBMIT` to 1 in the `main()` program's environment.
2. Specify the custom command in the command field of the hosts information file.

The following example describes a hosts information file that uses a custom submit command that is similar to RSH.

```
# HOSTS INFORMATION FILE
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
#   flag| hostname| slots|tmpDir| protocol | command
#
# Use /usr/local/bin/custom_cmd to start the workers.
1| teralab-x15 | 4 | /remote/users/tmp | RSH | /usr/local/bin/custom_cmd
1| teralab-x16 | 8 | /remote/users/tmp | RSH | /usr/local/bin/custom_cmd
```

Using the Hosts Information File to Specify CUSTOM Launch Protocol

Any customized launch protocol that is unknown to CDPL or that does not fit into any of the categories of CDPL-supported launch protocols can use the CUSTOM protocol.

When using the CUSTOM protocol, CDPL submits the job by prepending the user-defined submit command to the worker command, on the command line in a Linux shell.

The CUSTOM protocol is more flexible than the `custom submit` command. This protocol enables you to configure CDPL for sites having in-house DP submission commands that are defined for their compute farms. The CUSTOM protocol does not have the [three restrictions](#) required for using a custom submit command.

The CUSTOM protocol is used for sites that prefer their wrapper scripts to submit jobs to their compute farm. It is used to support or known compute

farms such as LSF and SGE, and even for compute farms that CDPL does not have a native support.

The following example describes a hosts information file that uses the CUSTOM protocol to submit the workers to a compute farm.

```
# HOSTS INFORMATION FILE
#   Each valid host definition occupies an entire line.
#   Line with '#' as the first character is a comment line.
#   Format of each line:
# flag| hostname| slots|tmpDir| protocol | command
#
# Use CUSTOM protocol to start the workers.
1| | 4| /remote/users/tmp | CUSTOM | /path/myfarm_submit_command <options>
```

Note:

1. Provide the full pathname of the submit command.
2. A potentially noticeable difference between the CUSTOM protocol and the SGE and LSF protocols is that *users might have to add the proper command submit option to carry the `main()` program's environment variables over to the DP worker side*. For example, if the CUSTOM protocol is used to submit a command to the SGE farm, the `qsub -V` option is required.
3. To use the CUSTOM protocol to submit jobs on the SGE farm, in most end-user environments, the `qsub -b` option should be given a value of `y`, whereas when the SGE protocol is used, the `qsub -b` option should be given a value of `n`.
4. To use the CUSTOM protocol to submit a job on farms that allow double quotes for command line options, in older CDPL releases, add the escape character, that is, the `\` symbol in front of the double quotes.

For example, to submit a job on the LSF farm:

- In the host configuration file:

```
1 || -1 | /tmp | CUSTOM | bsub -q bnormal -R \"arch==glinux &&
(os_version==CS6_0 || os_version==SUSE11_0)\"
```

- In a program:

```
char const * const cmd = "bsub -q bnormal -R \"'arch==glinux &&  
(os_version==CS6_0 || os_version==SUSE11_0) '\"";  
  
cdplerr = cdplAddHost(ms, NULL, "/tmp", CDPL__CUSTOM, -1, cmd);
```

3

Worker Information File

The worker information file allows EDA tool users to specify *attributes* of DP workers. Specifically, the worker information file would include the following information:

- The *type* of worker required for DP.
- The *name*, *location*, and *command line options* of the worker executable.
- The *number* of workers for each type of DP worker, including the minimum number of workers to start with and the maximum number of workers allowed at any given time.

The worker information file is an ASCII file. Single-line comments are allowed and must be prefixed with '#'. Every line in the file has the following format.

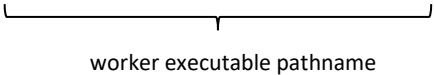
```
<worker_type>; <max_workers>; <min_workers>; <worker_executable and  
parameters>
```

Note that semicolon is used as the delimiter in the worker configuration file.

Field	Value	Purpose
<i>Worker type</i>	Integer	The type of worker for executing the tasks of the application. It can be any integer number other than those reserved for CDPL's thinworker (20000 - 20999) and thinrelay (50000 - 50999). This integer must be mutually exclusive for <i>multiple applications</i> (that is, EDA tools) as workers used in a single DP run. In other words, ensure that each entry has a <i>unique</i> integer for the given type of worker.
<i>Max workers</i>	Integer	The maximum number of DP workers. If there are pending tasks to be processed, CDPL attempts to start as many workers as required, up to the maximum value specified in this field. The value specified for this field should not surpass the value that is specified for the compute slots field defined in the hosts information file in Chapter 2, Hosts Information File .
<i>Min workers</i>	Integer	The minimum number of workers required for successful operation. It should be at least '1'.
<i>Worker executable</i>	String - a valid executable program or script	The full path of the worker executable. The worker executable can be prepended with a wrapper script that can be very useful in providing visibility into worker operation. A sample wrapper script (START_WORKER) is provided in the "tutorials" subdirectory of the CDPL SDK.
<i>parameters</i>	String	Application specific parameters required by the worker executable. The worker executable and parameters form the command line that will be executed.

Here is a sample worker information file:

```
# Worker Information file
#
10; 1; 1; /home/user1/cdpl/tutorial/START_WORKER /home/user1/cdpl/tutorial/countcomments -worker
```



worker executable pathname



worker command line options

4

Docker Configuration File

To turn on Docker container support in DP, a Docker configuration file must be provided to the CDPL runtime. CDPL provides a Docker configuration template file that users must modify and add information to it.

The docker configuration file format has the following keys:

- **Image:** Specifies what the docker image is. This follows the Docker client `docker run image option syntax <registry>/<docker image>`.
- **Options:** Specifies the Docker options. This follows the `docker run option syntax`.

The following is an example of a configuration file:

```
# testing this comment

image      pst73:443/centos7-fc_2020.09-sp1

# testing this second comment and space above

options --volume=/var/lib/sss/pipes:/var/lib/sss/pipes:shared

options --volume=/global:/global:shared

options --volume=/remote:/remote:shared

options --volume=/depot:/depot:shared
```

In this example, option

`--volume=/var/lib/sss/pipes:/var/lib/sss/pipes:shared`

is an actual option that can be passed to `docker run`.

Note:

1. To run Docker containerized applications in a Docker environment, you must have the correct Docker access permission and setup. This is a prerequisite for CDPL Docker support that you should work out with the help of your IT admins or cloud vendors.
2. In the docker image value and options values, it is assumed that there are no whitespaces. For example, the following would be invalid:

```
image      pst73:443/  centos7-fc_2020.09-sp1
```

3. Entries of multiple options are allowed.
4. Comments starting with a “#” are allowed in the configuration file.

5

Debugging Distributed Environments

Distributed processing (DP) environments depend on the consumer configuration of their infrastructure. Some of these configurations can be different from the internal environment configuration of Synopsys.

To aid field engineers and R&D in running distributed processing in the consumer environment, CDPL provides a few utilities and methods that help in analyzing and validating the user's environment and capturing the detailed run information.

Besides the descriptions of CDPL DP utilities that follow, if required, refer to the *CDPL Farm Interface Troubleshooting Guide*.

DPCHECK – Infrastructure Validation

`dpcheck` is a utility distributed with the CDPL source code and SDK. The utility script is used to ensure that the SGE/LSF/PBS/RTDA/NB/SLURM infrastructure is ready to accept jobs. It also ensures that RSH/SSH/SH/ protocols are working properly on the specified hosts. This script is available in the `$CDPL_HOME/bin` directory. Run the `dpcheck -h` command to get the correct usage.

The following displays the output of the `dpcheck -h` command.

```
% dpcheck -h
```

Usage:

```
dpcheck -h|--help
```

```
dpcheck -m {rsh|sge|lsf|ssh|sh|pbs|rt-da|nb|slurm}
```

```
dpcheck -host {host file}
```

CDPL needs `rsh|ssh|lsf|sge|sh|pbs|rt-da|nb|slurm` to be set correctly for launching workers.

This program allows you to validate the transport settings.

The following checks are performed with the options below.

-m:

`rsh|ssh`: check if localhost login is allowed (passwordless)

`sge|lsf|pbs|rt-da|nb|slurm`: check for environment settings, submit host and default

queues

automount check on localhost.

-host:

This checks the availability of the hosts listed in the host file.

Also, all the checks done with the `-m` option are performed here.

-i hostlist.txt

Generates a CDPL host file from an input text file containing a list of hosts. The mode must be set to `rsh,ssh` or `sh` to use this option. The input file `hostlist.txt` contains 2 columns. The first column is either a 1 or 0 indicating if the host is enabled or disabled. The second column is the name of the host.

-o hostfile

The `-o` option is an optional parameter used to specify the name of the CDPL host file that is generated from an input host list. The host list is the input file provided with the `-i` option. When an input host list is provided, but the `-o` option is not used, the host data is output to the default file name `"hostname.mode.hosts"`.

An exit code of 0 signifies the proper environment.

stdout contains additional information. All errors are reported to stderr.

Checking a Hosts Information File With dpcheck

To check the hosts information file configuration, use the `dpcheck -host <hostfile>` command. In the following example, the `dpcheck` utility is used to check the configuration of an RSH hosts information file.

```
% dpcheck -host hosts.rsh
automount process is running at localhost.
Valid hostfile found for DP run.
Valid [rsh] environment found for DP runs.
    : t3amd03: Fri Nov  5 10:20:33 PDT 2010
    : t3amd01 is alive
    : t3amd02 is alive
    : t3amd03 is alive
    : t3amd01: Fri Nov  5 10:20:33 PDT 2010
    : t3amd02: Fri Nov  5 10:20:34 PDT 2010
    : t3amd03: Fri Nov  5 10:20:34 PDT 2010
```

Checking SGE Settings With dpcheck

To check if SGE settings are correctly configured for a given user ID, login to your interactive shell and run the following command:

```
% dpcheck -m sge

Valid [sge] environment found for DP runs.
    : teralab-x4150 is a valid submit host
```

Checking LSF Settings With dpcheck

To check whether the LSF environment is similar to the SGE environment, run the following command:

```
% dpcheck -m lsf
```

```
Valid [lsf] environment found for DP runs.  
: Default queue is [bnormal]
```

Checking PBS Settings With dpcheck

To check whether the PBS environment is similar to LSF and SGE, run the following command:

```
% dpcheck -m pbs
```

```
Valid [pbs] environment found for DP runs.  
: t3syn07 is a valid submit host
```

Checking RTDA Settings With dpcheck

To check whether the RTDA environment is similar to LSF and SGE, run the following command:

```
% dpcheck -m rtda
```

```
Valid [rtda] environment found for DP runs.  
: NC_QUEUE is vnclogic  
: Submit command: /remote/fsd/rtda/2011.03/linux/scripts/nc submit  
: t3syn03 is a valid submit host
```

Checking Netbatch Settings With dpcheck

To check whether the Netbatch environment is similar to LSF and SGE, run the following command:

```
% dpcheck -m nb
```

```
Valid [nb] environment found for DP runs.
```

Many simple errors can be detected by using this utility.

Example: An Erroneous SGE Configuration

Here is an example of a machine that is not configured to run SGE on a given farm.

```
% dpcheck -m sge
```

Errors in [sge] check:

**error running /remote/sge/default/bin/lx24-amd64/qconf: error:
denied: host "snpsamdNNN" is neither submit nor admin host**

DPVALIDATE – Infrastructure Validation

`dpvalidate` is a utility distributed with the CDPL source code and SDK. This utility script is used to check the readiness of the environment for CDPL. This script is available in the `$CDPL_HOME/bin` directory. Run `dpvalidate` to get the correct usage.

Here is the usage of `dpvalidate`.

```
% dpvalidate
```

```
Setting launch timeout to 30 seconds!
```

```
Usage: dpvalidate [-c -p port -h host [-m message]]
```

```
dpvalidate -l {rsh|ssh|qsub|bsub|sh}
```

`dpvalidate` indicates whether the LSF or SGE farm environment is ready to use CDPL. This is because it simulates a CDPL connection to the farm using the default `bsub` (LSF) or `qsub` (SGE) command.

Here is an example of the output of the `dpvalidate` command when checking an LSF farm.

```
% dpvalidate -l bsub

Setting launch timeout to 30 seconds!
Platform LSF HPC 7 Update 6, Sep 04 2009
Copyright 1992-2009 Platform Computing Corporation

My cluster name is cdpl
My master name is t3master01
CDPL safe!

DPMasterHost  DPClient    Slot  Return_Code
t3sys03       t3sys01 1    9080 SUCCESS [bsub], bsub
```

The following example shows the output of `dpvalidate` when checking an SGE farm.

```
% dpvalidate -l qsub

Setting launch timeout to 30 seconds!
CDPL safe!

DPMasterHost  DPClient    Slot  Return_Code
t3sys03       t3sys06 1    9080 SUCCESS [qsub], qsub
```

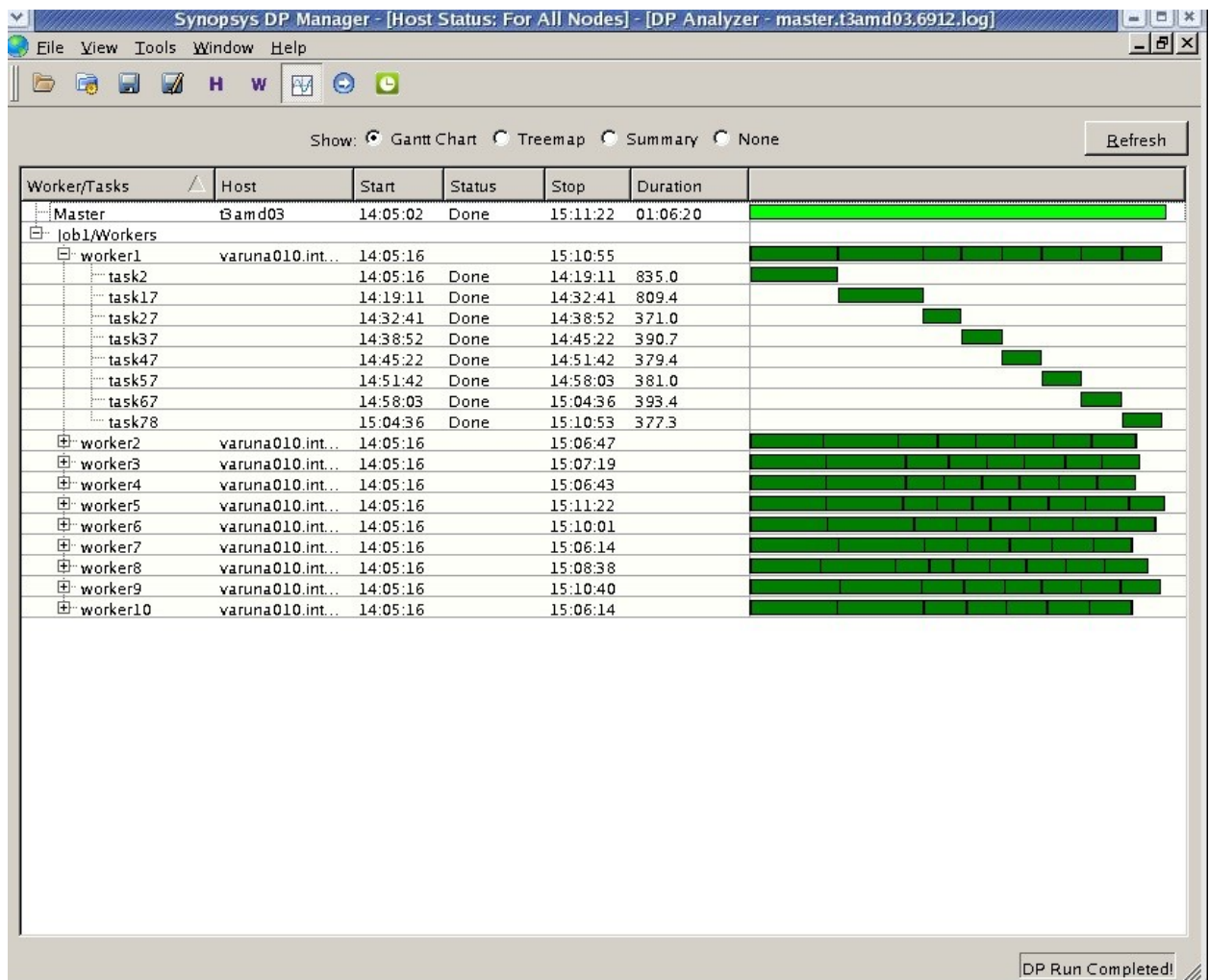
If `dpvalidate` reports errors when connecting to the LSF, SGE, PBS, or RTDA farm, identify the site-specific requirements of the farm and integrate CDPL with them. This might involve integrating CDPL with site-specific job submission commands or security methods.

DP Analyzer

DP Analyzer is a widget provided within the CDPL DP Manager GUI as part of the SDK release. It is a graphical screen that allows you to visually inspect

a running or finished job and analyze certain characteristics of the job and tasks.

The following figure shows DP Analyzer Gantt chart view of a completed job. An important characteristic of this view is the white space in the task timeline of each worker. This indicates the time during which the worker did not perform a task assigned by the DP scheduler. This type of behavior is unexpected and further analysis should be done to determine why the worker did not actively perform a task during this time.



Log Files

A typical CDPL application will generate several DP (distributed processing) system log files in addition to the application log files. The DP system log files are informative and useful when investigating problems with a DP run.

The following log files are created during a DP run; “master” and “worker” are default prefixes to the log files but can be replaced by the EDA tool with application specific prefixes:

```
master.<hostname>.<pid>.log
worker.W1.<hostname>.<pid>.log
...
worker.Wn.<hostname>.<pid>.log
```

The number of worker log files will be equal to the number of workers that run during the job.

The master log file provides most of the information about the DP run. Always check the master log file first when investigating problems.

CDPL Release Version

The CDPL release version is present in the header section of each log file. For example,

```
@@start_of_log@@
time: 2020/05/22 11:39:49 PDT -0700
version: Q_2019.12_SP1_PROD_CDPL
system: Linux 2.6.32-754.11.1.el6.x86_64 #1 SMP Tue Feb 26 15:38:56
UTC 2019
...
```

Note: The CDPL release version is required when reporting issues to CDPL development team.

Log Viewing Utilities: log2tty and log2html

You can find these utilities in the `$CDPL_HOME/bin` directory. They translate timestamps and filter output from the master and worker log files into a more readable format. Use `log2tty` when viewing the output in a command shell window. Use `log2html` to generate output to view in a web browser. The typical usage is:

```
log2tty <filename>
```

```
log2html <filename> > <filename.html>
```

where <filename> is a master or worker log file.

Sample Log From a Successful Run

The following example displays the output from `log2tty` when you view a master log file from a successful DP run. This DP run is for a small job with three workers and three tasks.

```
sh-3.00$
sh-3.00$ log2tty master.t3amd03.11985.log
PDT 2010/10/28 11:32:17.500484 -- time: 2010/10/28 11:32:17 PDT -0700
PDT 2010/10/28 11:32:17.500491 -- version: CDPL_E2010.09_SP1_PROD_BF2 1616494
PDT 2010/10/28 11:32:17.500496 -- system: Linux 2.6.9-67.ELsmp #1 SMP Wed Nov 7 13:56:44 EST 2007
PDT 2010/10/28 11:32:17.500500 -- processor: x86_64 64-bits
PDT 2010/10/28 11:32:17.500504 -- hostname: t3amd03
PDT 2010/10/28 11:32:17.500526 -- new server S975361824
PDT 2010/10/28 11:32:17.501058 -- new master M1
PDT 2010/10/28 11:32:17.501067 -- reading hostfile 'etc/hosts'
PDT 2010/10/28 11:32:17.516627 -- creating RSH farm
PDT 2010/10/28 11:32:17.516640 -- exporting CDPL_LOGDIR as '/remote/t3dev1/geiszler/cdpl/compat/samples/cdplgui'
PDT 2010/10/28 11:32:17.516644 -- exporting CDPL_TMPDIR as '/tmp'
PDT 2010/10/28 11:32:17.516649 -- exporting CDPL_HOME as '/remote/t3dev1/geiszler/cdpl'
PDT 2010/10/28 11:32:17.516652 -- exporting DISPLAY as ':1.0'
PDT 2010/10/28 11:32:17.516805 -- bcast dir is '/remote/us01home21/geiszler/.synopsys/cdpl'
PDT 2010/10/28 11:32:17.542220 -- new job J1 in M1
PDT 2010/10/28 11:32:17.542232 -- new worker type 10, min:1, max:4, cmd:'/remote/t3dev1/geiszler/cdpl/compat/samples/cdplgui/countcomments -worker'
PDT 2010/10/28 11:32:17.542238 -- new bin B1 in J1
PDT 2010/10/28 11:32:17.542252 -- new task T0 in B1 (prio 0)
PDT 2010/10/28 11:32:17.542259 -- new task T1 in B1 (prio 0)
PDT 2010/10/28 11:32:17.542264 -- new task T2 in B1 (prio 0)
PDT 2010/10/28 11:32:17.542283 -- starting job J1
PDT 2010/10/28 11:32:17.542365 -- started worker W1 (as C1) in bin B1
PDT 2010/10/28 11:32:17.542383 -- started worker W2 (as C2) in bin B1
PDT 2010/10/28 11:32:17.542400 -- started worker W3 (as C3) in bin B1
PDT 2010/10/28 11:32:33.053556 -- W3 is 't3amd03:57859'
PDT 2010/10/28 11:32:33.053592 -- assigning task T0 to worker W3
PDT 2010/10/28 11:32:33.109431 -- worker W3 completed task T0
PDT 2010/10/28 11:32:33.109435 -- stat T0: utime=1 stime=1
PDT 2010/10/28 11:32:33.109440 -- statm T0: vsize=29232 rss=944 shared=772 text=272 data=20792
PDT 2010/10/28 11:32:33.109469 -- assigning task T1 to worker W3
PDT 2010/10/28 11:32:33.116319 -- W2 is 't3amd02:40594'
PDT 2010/10/28 11:32:33.116343 -- assigning task T2 to worker W2
PDT 2010/10/28 11:32:33.141454 -- W1 is 't3amd01:50858'
PDT 2010/10/28 11:32:33.170510 -- worker W2 completed task T2
PDT 2010/10/28 11:32:33.170514 -- stat T2: utime=0 stime=2
PDT 2010/10/28 11:32:33.170518 -- statm T2: vsize=29232 rss=944 shared=772 text=272 data=20792
PDT 2010/10/28 11:32:33.208484 -- worker W3 completed task T1
PDT 2010/10/28 11:32:33.208488 -- stat T1: utime=0 stime=0
PDT 2010/10/28 11:32:33.208492 -- statm T1: vsize=29232 rss=964 shared=792 text=272 data=20792
PDT 2010/10/28 11:32:33.558410 -- freeing job J1
PDT 2010/10/28 11:32:33.558428 -- stopping job J1
PDT 2010/10/28 11:32:33.561830 -- freeing server S975361824
PDT 2010/10/28 11:32:33.562325 -- killing server
PDT 2010/10/28 11:32:33.562422 -- freeing master M1
PDT 2010/10/28 11:32:33.562675 -- freeing farm
sh-3.00$
```

Sample Log With Errors

The following example is the log file output from `log2tty` of a DP run that had errors and did not complete. In this example, the error occurred because the worker executable failed to run on the remote system. When such an error occurs check whether the worker executable file is available on the remote system and that you can run it there.

```
sh-3.00$
sh-3.00$ log2tty master.t3amd03.24312.log
PDT 2010/10/28 11:50:00.271711 -- time: 2010/10/28 11:50:00 PDT -0700
PDT 2010/10/28 11:50:00.271719 -- version: CDPL_E2010.09_SP1_PROD_BF2 1616494
PDT 2010/10/28 11:50:00.271723 -- system: Linux 2.6.9-67.ELsmp #1 SMP Wed Nov 7 13:56:44 EST 2007
PDT 2010/10/28 11:50:00.271730 -- processor: x86_64 64-bits
PDT 2010/10/28 11:50:00.271733 -- hostname: t3amd03
PDT 2010/10/28 11:50:00.271755 -- new server S1534418917
PDT 2010/10/28 11:50:00.272440 -- new master M1
PDT 2010/10/28 11:50:00.272449 -- reading hostfile 'hosts'
PDT 2010/10/28 11:50:00.301588 -- creating RSH farm
PDT 2010/10/28 11:50:00.301615 -- exporting CDPL_LOGDIR as '/remote/t3dev1/geiszler/cdpl/compat/samples/cdplgui'
PDT 2010/10/28 11:50:00.301619 -- exporting CDPL_TMPDIR as '${CDPL_HOME}'
PDT 2010/10/28 11:50:00.301625 -- exporting CDPL_HOME as '/remote/t3dev1/geiszler/cdpl'
PDT 2010/10/28 11:50:00.301629 -- exporting DISPLAY as ':1.0'
PDT 2010/10/28 11:50:00.301721 -- bcast dir is '/remote/us01home21/geiszler/.synopsys/cdpl'
PDT 2010/10/28 11:50:00.304843 -- new job J1 in M1
PDT 2010/10/28 11:50:00.304865 -- new worker type 10, min:1, max:4, cmd:'/remote/t3dev1/geiszler/cdpl/compat/samples/cdplgui/countcomments -worker'
PDT 2010/10/28 11:50:00.304871 -- new bin B1 in J1
PDT 2010/10/28 11:50:00.304903 -- new task T0 in B1 (prio 0)
PDT 2010/10/28 11:50:00.304911 -- new task T1 in B1 (prio 0)
PDT 2010/10/28 11:50:00.304921 -- new task T2 in B1 (prio 0)
PDT 2010/10/28 11:50:00.304940 -- starting job J1
PDT 2010/10/28 11:50:00.305041 -- started worker W1 (as C1) in bin B1
PDT 2010/10/28 11:50:00.305058 -- started worker W2 (as C2) in bin B1
PDT 2010/10/28 11:50:00.305074 -- started worker W3 (as C3) in bin B1
PDT 2010/10/28 11:55:00.354184 -- W1 has timed out
PDT 2010/10/28 11:55:00.354244 -- W2 has timed out
PDT 2010/10/28 11:55:00.354271 -- W3 has timed out
PDT 2010/10/28 11:55:00.354294 -- 3 workers have timed out in B1
PDT 2010/10/28 11:55:00.354307 -- worker command for B1 appears to be invalid
PDT 2010/10/28 11:55:00.354314 -- offending command is '/remote/t3dev1/geiszler/cdpl/compat/samples/cdplgui/countcomments -worker'
PDT 2010/10/28 11:55:00.354320 -- B1 bailing out
PDT 2010/10/28 11:55:00.354334 -- killing job J1
PDT 2010/10/28 11:55:00.615138 -- freeing job J1
PDT 2010/10/28 11:55:00.618111 -- freeing server S1534418917
PDT 2010/10/28 11:55:00.618447 -- killing server
PDT 2010/10/28 11:55:00.618533 -- freeing master M1
PDT 2010/10/28 11:55:00.619349 -- freeing farm
sh-3.00$
```

Managing DP Log Files: dppurge

The DP log files will accumulate over time. Unless they are periodically deleted, they could eventually harm the system's performance. The files that

accumulate over many DP runs are the master and worker log files, and also the broadcast files in the `$HOME/.synopsys/cdpl` directory.

CDPL provides the `dppurge` utility to find and remove files from old DP jobs. The `dppurge` utility is available in the `$CDPL_HOME/bin` directory. Here is the `dppurge` command line usage:

```
dppurge -h
```

Usage:

```
dppurge [-k numjobs] [-b bcastdir]
```

```
dppurge -h|--help
```

This program removes files from old DP jobs. The broadcast files, master log files, and worker log files are removed. The following options are available:

`-k numjobs`

Specifies the number of jobs to keep. By default, files from all jobs are removed. This is equivalent to `-k 0`.

`-b bcastdir`

Specifies the broadcast directory used to locate the DP files.